

NOVSMM Enterprise Architecture Migration

Nuevo Rol

A partir de este momento deja de actuar únicamente como desarrollador.

Quiero que asumas el rol de:

- Chief Technology Officer (CTO)
- Software Architect
- Senior Backend Engineer
- DevOps Engineer
- Security Engineer
- Site Reliability Engineer (SRE)

Tu prioridad ya no será agregar nuevas funcionalidades.

Tu prioridad será construir la infraestructura técnica definitiva de NOVSMM para producción.

Objetivo General

Quiero convertir NOVSMM en una plataforma SaaS Enterprise.

El objetivo es que la arquitectura pueda soportar desde cientos hasta millones de usuarios sin tener que rehacer el proyecto dentro de unos meses.

No quiero soluciones temporales.

No quiero código de demostración.

No quiero implementaciones rápidas.

Quiero una arquitectura limpia, escalable, mantenible y preparada para crecimiento a largo plazo.

Regla Principal

Antes de modificar cualquier módulo existente:

- Analiza su funcionamiento.
- Verifica compatibilidad.
- Evita romper funcionalidades.
- Mantén compatibilidad con el frontend.
- Si es necesario migrar una funcionalidad, hazlo de forma gradual.
- No elimines código estable hasta que la nueva implementación haya sido validada.

La estabilidad del sistema es prioritaria.

Arquitectura Objetivo

Toda la plataforma deberá evolucionar hacia una arquitectura similar a esta:

Internet



Cloudflare



Nginx



Next.js



PostgreSQL



Redis



Background Jobs



Notification Service



Payment Services



APIs SMM

Cada componente debe tener una responsabilidad clara.

1. Base de Datos

Actualmente el proyecto utiliza SQLite.

Quiero migrar completamente hacia PostgreSQL.

Objetivos:

- Actualizar Prisma.
- Crear migraciones profesionales.
- Optimizar índices.
- Optimizar consultas.
- Evitar consultas N+1.
- Configurar Connection Pooling.
- Mantener compatibilidad durante la migración.
- Documentar cada cambio.

No quiero perder información.

No quiero romper la aplicación.

2. Redis

Integrar Redis como parte fundamental de la plataforma.

Redis deberá utilizarse para:

- Caché.
- Rate Limiting.
- Sesiones (cuando aplique).
- WebSockets.
- Colas.
- Locks distribuidos.
- Datos temporales.

No instalar Redis únicamente por instalarlo.

Cada implementación debe tener una justificación técnica.

3. Background Jobs

Separar completamente todas las tareas pesadas del flujo principal.

Crear un sistema de colas para:

- Emails.

- Notificaciones.
- Procesamiento de pagos.
- Webhooks.
- Sincronización de proveedores.
- Reintentos automáticos.
- Limpieza de datos.
- Exportaciones.
- Procesamiento masivo.

Implementar:

- Retries automáticos.
- Dead Letter Queue.
- Logs.
- Monitoreo.
- Control de errores.

4. Seguridad

Quiero endurecer completamente la seguridad.

Implementar:

- Middleware global.
- CSP (Content Security Policy).
- Rate Limiting.
- Protección CSRF.
- Sanitización de entradas.
- Headers de seguridad.
- Validación estricta.
- Protección contra fuerza bruta.
- Lock temporal de cuentas.
- Auditoría completa.
- Gestión de dispositivos.
- Historial de sesiones.
- Rotación segura de secretos.

Las llaves privadas nunca deben exponerse al frontend.

5. Arquitectura Backend

Revisar completamente:

- Servicios.
- Controladores.
- Hooks.
- API Routes.

- Server Actions.
- Utilidades.
- Librerías.

Eliminar duplicación.

Reducir complejidad.

Mejorar mantenibilidad.

Aplicar principios SOLID donde tenga sentido.

6. APIs

Revisar todas las APIs existentes.

Validar:

- Seguridad.
- Errores.
- Tipado.
- Respuestas.
- Validaciones.
- Rendimiento.

Todas deben seguir una estructura uniforme.

7. Performance

Optimizar toda la plataforma.

Revisar:

- Renderizado.
- Lazy Loading.
- Dynamic Imports.
- Cache.
- Imágenes.
- Bundle.
- Consultas.
- Revalidación.
- Streaming.

Objetivo:

Máxima velocidad.

8. Observabilidad

Implementar un sistema completo de monitoreo.

Registrar:

- Errores.
- Logs.
- Latencia.
- Uso de memoria.
- CPU.
- Errores de base de datos.
- Errores de APIs.
- Fallos de WebSockets.
- Fallos de pagos.

Quiero poder detectar problemas antes que los usuarios.

9. Backups

Implementar estrategia de respaldo.

- Backups automáticos.
 - Restauración.
 - Verificación.
 - Retención.
 - Recuperación ante desastres.
-

10. DevOps

Preparar la plataforma para producción.

Quiero que el proyecto quede preparado para desplegarse utilizando:

- Docker.
- Docker Compose.
- Nginx.
- PM2 (si se mantiene).
- Variables de entorno separadas por ambiente.
- Desarrollo.
- Testing.
- Producción.

Crear una estructura limpia y documentada.

11. Código

Revisar todo el proyecto.

Eliminar:

- Código muerto.
- Componentes duplicados.
- Hooks innecesarios.
- Dependencias sin uso.
- Archivos obsoletos.
- Configuraciones antiguas.

Quiero una base de código limpia.

12. Documentación

Generar documentación técnica completa.

Incluir:

- Arquitectura.
- Flujo de autenticación.
- Base de datos.
- APIs.
- Redis.
- Background Jobs.
- WebSockets.
- Seguridad.
- Deployment.
- Variables de entorno.
- Recuperación ante fallos.

Todo debe quedar preparado para que cualquier desarrollador pueda incorporarse al proyecto.

Forma de Trabajo

No implementes todo de una sola vez.

Primero realiza una auditoría técnica completa.

Después crea un plan de migración.

Divide el trabajo en fases.

Cada fase debe incluir:

- Objetivo.
- Riesgos.
- Cambios.
- Compatibilidad.
- Pruebas.
- Resultado esperado.

No continúes con la siguiente fase hasta validar completamente la anterior.

Objetivo Final

Cuando la migración termine quiero que NOVSM sea una plataforma SaaS Enterprise preparada para producción, segura, escalable, mantenible y optimizada, siguiendo estándares utilizados por empresas tecnológicas modernas.

Antes de implementar cualquier cambio importante, analiza su impacto sobre el sistema existente y prioriza siempre la estabilidad, la calidad del código y la escalabilidad a largo plazo.